

SymCorr: A Compiler Framework for Automated Construction and Efficient Evaluation of Lattice QCD Correlation Functions

...

Abstract

1 Introduction

1.1 Motivation

Lattice quantum chromodynamics (Lattice QCD) provides a first-principles, non-perturbative formulation of the strong interaction and has become the standard framework for studying hadron spectroscopy, hadronic matrix elements, and the interactions of multi-hadron systems. Modern spectroscopy calculations typically employ large variational operator bases to reliably isolate excited states, resonances, and near-threshold scattering channels. The quality of the extracted spectrum therefore depends critically on the construction of interpolating operators with good overlap onto the physical states of interest and on the efficient evaluation of the corresponding Euclidean correlation functions.

Among the available smearing techniques, the distillation framework introduced in Ref. [1] has emerged as one of the most successful approaches for spectroscopy calculations. By projecting the quark fields onto a low-dimensional Laplacian eigenvector basis, distillation provides a compact representation of quark propagation while preserving the flexibility to construct large and highly structured operator bases. It has enabled precision studies of excited hadrons, coupled-channel scattering, hybrid mesons, and multi-hadron systems that would otherwise be prohibitively expensive.

Despite these advantages, the software required to construct and evaluate correlation functions in the distillation framework remains largely hand-crafted. New operator constructions often require significant manual implementation effort, including symbolic Wick contractions, flavor and spin

bookkeeping, and the translation of analytical expressions into optimized tensor contractions. As operator bases become larger and more diverse, this manual workflow becomes increasingly difficult to maintain, verify, and extend.

This work introduces SYMCORR, a compiler framework that automates the construction and optimization of lattice QCD correlation functions within the distillation formalism. Rather than treating correlator construction as a collection of ad hoc tensor contractions, SYMCORR represents interpolating operators symbolically, performs automatic Wick contractions, identifies reusable computational structures, and generates optimized execution graphs suitable for efficient numerical evaluation.

1.2 Challenges in lattice QCD correlator construction

The numerical evaluation of hadronic correlation functions involves several distinct stages. Interpolating operators are first constructed with the desired quantum numbers, including flavor, spin, momentum, and lattice symmetry transformations. Correlation functions are then obtained by applying Wick’s theorem, producing a sum over quark contractions that must subsequently be expressed in terms of quark propagators.

Within the distillation formalism these propagators are replaced by perambulators and mode-space operator kernels. Although this factorization greatly reduces the computational cost of propagator inversions, it introduces a new layer of algebraic complexity. The resulting expressions consist of large collections of tensor contractions whose structure depends on the flavor content, spin couplings, momentum projections, derivatives, and color contractions appearing in the interpolating operators.

For realistic spectroscopy calculations involving dozens or hundreds of operators, manually deriving and implementing these contractions is time-consuming and prone to error. Small modifications to an operator basis often require extensive changes throughout the implementation, while redundant sub-expressions are difficult to identify by hand. These challenges are particularly severe for multi-hadron systems, where the number of Wick contractions grows rapidly with the number of quark fields.

Consequently, there is a need for software capable of representing interpolating operators symbolically, automatically performing Wick contractions, exploiting the algebraic structure of distillation, and producing optimized computational graphs for numerical evaluation.

1.3 Existing software

There are several existing software packages that provide tools for constructing the interpolating operators in the subduced representations of the lattice symmetry groups such as Multi Hadron Interpolators (MHI) [2] and OpTion[3]. Following this the next step is to perform the Wick contractions and evaluate the resulting expressions. There are several software packages that automate this process and also various algorithms have been proposed to identify the redundancies and optimize the evaluation of the resulting expressions such as those proposed in [4, 5]. However, these packages are not designed to work with the distillation framework and do not provide the necessary abstractions to represent the distillation formalism.

1.4 Contributions of SymCorr

In this work we present SYMCORR, a domain-specific compiler framework that automates the construction and optimization of lattice QCD correlation functions within the distillation formalism. The main contributions of this work are as follows: The principal contributions of this work are

- A symbolic intermediate representation for lattice QCD interpolating operators that naturally captures flavor, spin, derivatives, momentum projection, and color structure.
- An automatic Wick contraction engine that transforms operator products into sums of quark-line contraction graphs without manual derivation.
- A compiler framework that lowers symbolic expressions into optimized distillation-space execution graphs by identifying reusable computational structures.
- A backend capable of generating efficient numerical evaluation routines that separate symbolic operator construction from architecture-dependent execution.
- A modular software design that enables rapid development of new operator bases while reducing implementation effort and the likelihood of algebraic errors.

2 Overview of SymCorr

The construction of lattice QCD correlation functions naturally consists of a sequence of symbolic and numerical transformations. Traditionally, these stages are implemented manually, with the symbolic derivation of correlation functions tightly coupled to their numerical realization. As operator bases become larger and more sophisticated, this workflow becomes increasingly difficult to maintain, verify, and extend.

SYMCORR addresses this problem by treating correlator construction as a domain-specific compiler. Rather than directly translating interpolating operators into numerical routines, the compiler progressively lowers a high-level symbolic description into an optimized execution plan through a sequence of well-defined compilation stages. This separation of concerns allows symbolic manipulations, algebraic optimizations, and backend-specific execution strategies to be developed independently.

The overall compilation pipeline is illustrated in Fig. ?? . The process begins with a symbolic representation of interpolating operators, where the fundamental building blocks of lattice QCD—quark fields, gauge links, Dirac matrices, derivatives, and index structures—are represented independently of any numerical backend. The symbolic expressions are subsequently processed by an automatic Wick contraction engine, producing the complete set of fermionic contractions required to evaluate the correlation function.

The contracted expressions are then lowered into a logical execution plan that represents the computation as a collection of backend-independent operations. At this stage, the compiler identifies reusable computational structures and algebraic dependencies while remaining agnostic to the underlying numerical implementation. The logical plan is subsequently transformed into a physical execution plan, where abstract operations are mapped onto concrete numerical primitives appropriate for the chosen backend, such as distillation kernels, perambulators, and tensor contractions.

Finally, the runtime system executes the optimized physical plan on numerical lattice data to produce the desired correlation functions. By separating symbolic operator construction from numerical evaluation, SYMCORR provides a modular and extensible framework that simplifies the implementation of new interpolating operators while enabling backend-specific optimizations without modifying the underlying physics description.

3 Symbolic Representation

The frontend of SYMCORR provides a symbolic language for constructing lattice QCD interpolating operators. Rather than directly generating numerical tensor contractions, operators are first represented as symbolic expressions that closely resemble their mathematical definitions. This symbolic representation preserves the flavor, spin, color, spatial, momentum, and derivative structure of the operator while remaining completely independent of the numerical backend. Consequently, algebraic manipulations such as Wick contractions, expression simplification, and compiler optimizations can be performed before any numerical implementation is introduced.

Internally, every symbolic expression is represented as an abstract syntax tree (AST). The leaf nodes correspond to elementary lattice QCD objects such as quark fields, gamma matrices, gauge links, derivatives, momentum projections, and tensor objects, while internal nodes represent algebraic operations including multiplication, summation, tensor contraction, and operator composition. This tree-based representation preserves the complete symbolic structure of the interpolating operator and forms the intermediate representation on which subsequent compiler passes operate.

In its most general form, an interpolating operator may be regarded as an algebraic composition of elementary symbolic objects,

$$\mathcal{O} = \mathcal{C} \left[\prod_{i=1}^N \Phi_i \right], \quad (1)$$

where

$$\Phi_i \in \{q, \bar{q}, \Gamma, U, D, P, \epsilon, \delta, \dots\}, \quad (2)$$

and $\mathcal{C}$ denotes the complete contraction of color, spin, flavor, spatial, and other auxiliary indices required to project the operator onto the desired quantum numbers. The elementary symbolic objects represented by SYMCORR are described below.

3.1 Quark fields

The fundamental fermionic objects are quark and antiquark fields,

$$q_f^{a\alpha}(x), \quad \bar{q}_f^{a\alpha}(x), \quad (3)$$

where f denotes the flavor index, a the color index, α the Dirac spin index, and x the spacetime coordinate. These symbolic fields constitute the

elementary degrees of freedom from which all interpolating operators are constructed.

Since the frontend manipulates symbolic objects rather than numerical fields, the representation is independent of the underlying fermion discretization, allowing the same symbolic description to be compiled for different numerical backends.

3.2 Indices

Every symbolic object carries a collection of indices describing its transformation properties. Depending on the object, these include color, spin, flavor, spacetime, momentum, derivative, and displacement indices. Throughout the compilation process these indices remain symbolic, allowing the compiler to perform consistency checks, identify valid tensor contractions, and preserve the algebraic structure of the operator until numerical code generation.

3.3 Dirac matrices

Dirac matrices are represented as symbolic operators acting on spinor indices. They may be freely combined with quark fields to construct arbitrary fermion bilinears and higher-order interpolating operators. Treating Dirac matrices symbolically enables the compiler to manipulate operator expressions without prematurely expanding the associated spin algebra.

3.4 Gauge transporters

Gauge transporters are represented by symbolic Wilson lines,

$$U(P) = \prod_{\ell \in P} U_{\ell}, \quad (4)$$

where P denotes an arbitrary lattice path. These objects ensure gauge covariance for non-local interpolating operators and displaced quark fields. Because the path itself is represented symbolically, arbitrary displacement patterns can be incorporated without modifying the compiler.

3.5 Covariant derivatives

Covariant derivatives are represented as symbolic operators acting on quark fields. They may be combined to construct arbitrary derivative structures and operators carrying non-zero orbital angular momentum. Their symbolic

representation preserves the derivative structure throughout the compilation pipeline before being lowered to the corresponding backend-specific implementation.

3.6 Momentum projections

Momentum projection operators are represented by symbolic phase factors,

$$P_{\mathbf{p}}(x) = e^{i\mathbf{p}\cdot\mathbf{x}}, \quad (5)$$

which may act on arbitrary symbolic expressions. By retaining momentum projections symbolically, SYMCORR preserves translational properties and momentum conservation until the numerical execution stage.

3.7 Tensor objects

Many interpolating operators require additional tensors to contract color, spin, flavor, or lattice symmetry indices. These include the Kronecker delta, the Levi-Civita tensor,

$$\delta_{ij}, \quad \epsilon_{abc}, \quad (6)$$

as well as arbitrary user-defined symbolic tensors. These objects are treated uniformly within the symbolic representation and participate in the same algebraic manipulation and contraction framework as the remaining symbolic objects.

3.8 Interpolating operators

Interpolating operators are obtained by composing the elementary symbolic objects introduced above through algebraic operations represented in the AST. The resulting expressions may describe local or non-local operators, mesons, baryons, tetraquarks, pentaquarks, hybrid operators, or arbitrary multi-hadron systems.

For example, a generic meson interpolating operator may be written as

$$\mathcal{O}_M(x) = \bar{q}_f(x) \Gamma D^{(n)} U(P) P_{\mathbf{p}}(x) q_g(x), \quad (7)$$

whereas a baryon interpolating operator takes the form

$$\mathcal{O}_B(x) = \epsilon_{abc} \left(q_1^a(x) \Gamma_1 q_2^b(x) \right) \Gamma_2 q_3^c(x). \quad (8)$$

More complicated interpolators are constructed recursively by composing these elementary building blocks. Since the complete symbolic structure is preserved, the resulting expressions can subsequently undergo automatic Wick contraction, compiler optimization, and backend-specific lowering without requiring any manual algebraic manipulation.

4 Automatic Wick Contraction

Following the symbolic construction of the interpolating operators, SYM-CORR automatically applies Wick’s theorem to generate the corresponding correlation function. This stage transforms products of fermionic field operators into sums over quark propagator contractions while preserving the complete symbolic structure of the expression.

Unlike traditional workflows, where Wick contractions are derived manually for each new operator basis, the symbolic representation allows the contraction process to be fully automated. The output of this stage is a symbolic expression containing all valid contraction terms together with the associated fermionic permutation signs. These expressions form the input to the optimization stages described in the following sections.

4.1 Wick’s theorem

Consider a generic Euclidean correlation function

$$C(t) = \left\langle \mathcal{O}_{\text{sink}}(t) \mathcal{O}_{\text{source}}^\dagger(0) \right\rangle. \quad (9)$$

Applying Wick’s theorem replaces every quark–antiquark pair by the corresponding propagator,

$$q(x) \bar{q}(y) \rightarrow S(x, y), \quad (10)$$

and expresses the correlation function as a sum over all complete fermionic contractions. Since QCD conserves flavor, contractions are permitted only between quark and antiquark fields carrying identical flavor quantum numbers.

4.2 Contraction graph generation

The contraction engine first identifies every quark and antiquark field appearing in the symbolic expression. For each quark flavor, all complete

matchings between quark and antiquark fields are generated subject to flavor conservation. Every valid matching corresponds to one contraction graph.

If a correlation function contains n_f quark fields of flavor f , the number of possible flavor-preserving contractions is

$$N_{\text{contr}} = \prod_f n_f!. \quad (11)$$

The symbolic representation enables these contraction graphs to be generated independently of any numerical backend, allowing subsequent compiler passes to operate directly on the resulting algebraic structure.

4.3 Fermionic permutations

The contraction of Grassmann-valued fermion fields introduces a sign determined by the parity of the permutation required to bring contracted fields adjacent to one another. Consequently, every contraction graph is associated with a fermionic sign

$$(-1)^P, \quad (12)$$

where P denotes the parity of the corresponding permutation.

During contraction generation, SYMCORR explicitly tracks these permutations so that the correct fermionic phase accompanies every contraction term throughout the compilation pipeline.

4.4 Determinant representation

The antisymmetric nature of Grassmann fields allows many contraction expressions to be written compactly as determinants of quark propagators. Rather than explicitly enumerating every permutation, groups of contractions associated with identical fermion species can be represented by determinant expressions, leading to a substantial reduction in symbolic complexity.

Recent work [5] demonstrated that determinant representations together with polynomial identity testing can significantly reduce the number of independent determinants required for conventional lattice QCD correlators. Although this construction is highly effective for point propagators, the distillation framework introduces additional mode-space kernel structures that prevent its direct application.

In SYMCORR, the determinant formulation is retained as the underlying symbolic representation, while additional compiler optimizations are introduced to identify and eliminate redundant computations arising specifically within the distillation formalism. These optimizations are discussed in Section ??.

4.5 Expression simplification

The contraction stage generally produces expressions containing algebraically equivalent terms and redundant symbolic structures. Before lowering the expressions into the compiler intermediate representation, SYMCORR applies a collection of symbolic simplification rules.

These include the elimination of identical contraction terms, simplification of tensor contractions, canonical ordering of symbolic objects, and the application of algebraic identities involving Dirac matrices, color tensors, and momentum projections. Performing these simplifications symbolically reduces the size of the generated execution graph and improves the efficiency of subsequent optimization and numerical evaluation stages.

5 Compilation to the Distillation Representation

The symbolic expressions produced by the automatic Wick contraction stage are written in terms of quark propagators connecting contracted quark and antiquark fields. While these expressions are mathematically complete, they are not directly suitable for efficient numerical evaluation within the distillation framework. Instead, the quark propagators must be rewritten in terms of the elementary objects of distillation, namely the perambulators and mode-space operator kernels.

The purpose of this compilation stage is therefore to lower the symbolic Wick expressions into a backend-independent intermediate representation expressed entirely in terms of distillation primitives. During this transformation, SYMCORR identifies reusable operator kernels, factors propagators into their distillation representation, and constructs an execution graph that explicitly describes the numerical operations required to evaluate the correlation function. Since the compilation is performed symbolically, the resulting representation remains independent of the numerical backend while exposing opportunities for subsequent optimization and task scheduling.

The compilation pipeline implemented in this stage consists of the following steps:

1. Rewrite every quark propagator using the distillation decomposition.
2. Identify maximal mode-space operator kernels.
3. Construct reusable kernel descriptors.
4. Generate backend-independent contraction tasks.
5. Assemble the execution graph for numerical evaluation.

The following subsections describe each of these stages in detail.

5.1 Distillation formalism

Distillation introduces a low-rank approximation of the quark fields by projecting them onto a subspace spanned by the lowest eigenmodes of the gauge-covariant Laplacian operator. Where the laplacian operator is defined as

$$\Delta = \sum_{\mu} \left(2 - U_{\mu}(x) - U_{\mu}^{\dagger}(x - \hat{\mu}) \right), \quad (13)$$

and the eigenvectors $v_k(x)$ satisfy

$$\Delta v_k(x) = \lambda_k v_k(x), \quad (14)$$

where λ_k are the corresponding eigenvalues. The distillation operator is then

$$\square = \sum_{k=1}^{N_v} v_k(x) v_k^{\dagger}(y), \quad (15)$$

where N_v is the number of eigenvectors retained in the projection. The smeared quark fields are defined as

$$\tilde{q}(x) = \square q(x), \quad \tilde{\bar{q}}(x) = \bar{q}(x) \square. \quad (16)$$

5.2 Perambulators

The quark propagators in the distillation framework are replaced by perambulators, which are defined as

$$\tau_{kl}(t, t') = v_k^{\dagger}(t) S(t, t') v_l(t'), \quad (17)$$

where $S(t, t')$ is the quark propagator between timeslices t and t' , and $v_k(t)$ are the eigenvectors of the Laplacian operator at timeslice t . The perambulators encode the propagation of the smeared quark fields between different timeslices and are essential for constructing correlation functions in the

distillation formalism. They are typically precomputed and stored for reuse across multiple correlation function evaluations, significantly reducing the computational cost of the overall calculation.

5.3 Kernel identification

The distillation compilation stage identifies the mode-space operator kernels that arise from the contraction expressions. These kernels are defined as

$$K_{kl}(t) = v_k^\dagger(t) \Gamma D^{(n)} U(P) P_{\mathbf{p}}(t) v_l(t), \quad (18)$$

where Γ is a Dirac matrix, $D^{(n)}$ is a covariant derivative operator, $U(P)$ is a gauge transporter along path P , and $P_{\mathbf{p}}(t)$ is a momentum projection operator. The kernels capture the action of the interpolating operators in the mode space defined by the Laplacian eigenvectors. By identifying and reusing these kernels across different contraction terms, the compiler can reduce the number of unique computations required for evaluating the correlation functions, leading to improved performance and reduced memory usage.

5.4 Kernel recipes

5.5 Contraction tasks

6 Compiler Backend

6.1 Logical Execution Plan

```

Explain
  LogicalTask
  LogicalExecutionPlan

```

6.2 Static Analyses

```

DependencyAnalysis
  LifetimeAnalysis

```

6.3 Physical Lowering

```

Logical Plan
↓
Physical Execution Plan

```

6.4 Physical Instruction Set

Allocate

Load

Evaluate

Contract

Copy

Store

Free

Barrier

Wait

Event

Prefetch

6.5 Plan Verification

7 Optimization Framework

7.1 Planning Pipeline

7.2 Kernel Reuse

7.3 Memory Planning

7.4 Analytical Cost Model

8 Runtime System

8.1 Execution Context

8.2 Resource Manager

8.3 Storage Manager

8.4 Kernel Evaluation

8.5 Task Execution

8.6 Output Assembly

9 Worked Examples

9.1 Meson correlator

9.2 Baryon correlator

9.3 Multi-hadron correlator

10 Performance Evaluation

10.1 Compiler performance

Compilation time

10.2 Runtime performance

Execution time

10.3 Kernel reuse

10.4 Memory reduction

10.5 Scalability

11 Applications

Discuss

- Meson spectroscopy

- Baryons

- Multi-hadron operators

- Coupled-channel calculations

- Future large operator bases

12 Future Extensions

Profile-guided optimisation

- Multi-GPU

- Distributed execution

- Topology-aware scheduling

- Additional optimisation passes

13 Conclusions

References

- [1] Michael Peardon, John Bulava, Justin Foley, Colin Morningstar, Jozef Dudek, Robert G. Edwards, Balint Joo, Huey-Wen Lin, David G. Richards, and Keisuke Jimmy Juge. A Novel quark-field creation operator construction for hadronic physics in lattice QCD. *Phys. Rev. D*, 80:054506, 2009.
- [2] William Detmold, William I. Jay, Gurtej Kanwar, Phiala E. Shanahan, and Michael L. Wagman. Multiparticle interpolating operators in quantum field theories with cubic symmetry. *Phys. Rev. D*, 109:094516, May 2024.
- [3] Haobo Yan, Chuan Liu, Liuming Liu, and Yu Meng. Construction of general N -body lattice operators with arbitrary momenta. *JHEP*, 10:210, 2025.

- [4] Jie Chen, Robert G. Edwards, and Weizhen Mao. Graph contractions for calculating correlation functions in lattice qcd. In *Proceedings of the Platform for Advanced Scientific Computing Conference, PASC '23*, New York, NY, USA, 2023. Association for Computing Machinery.
- [5] Debsubhra Chakraborty, Piyush Srivastava, Arpith Kumar, and Nilmani Mathur. Nuclear correlation functions using first-principle calculations of lattice quantum chromodynamics. *Phys. Rev. D*, 110:114505, Dec 2024.